

Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (Raise Hand).

When instructor calls on you, click microphone icon to unmute. When you've finished speaking, *be sure to mute yourself again.*

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class for their own private use.

Releasing these recordings is strictly prohibited.

SEAS 8515

Data Engineering for AI

Professor Steven Cocke
Week 1: May 16, 2026

Agenda

- Introductions & Survey
- Course Overview & Admin Comments
- Questions about Python
- Python Foundations I
- Assignment Overview for This Week

Introductions

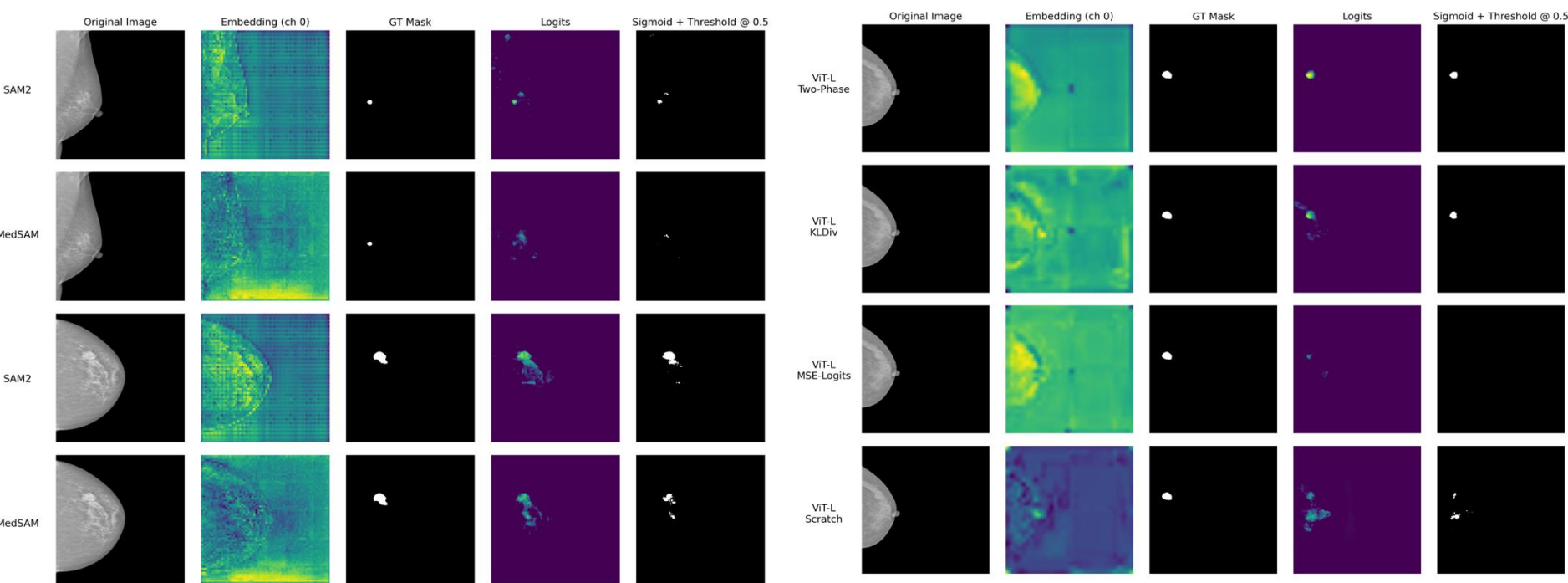
- Name
- Location
- Profession (current job)
- What do you hope to get out of this class?
- Have you settled on a Praxis topic yet?

About The Professor

- Bachelor's in System's Engineering (UVA)
- Master's in Data Science (SMU)
- Doctorate in Artificial Intelligence and Machine Learning (GWU)
- Teaching at GWU since 2023 - courses in software engineering, data engineering (this one), Python programming, and AI/Machine learning
- Experience: Consultant -> Data Engineer -> Data Analyst -> Data Scientist -> Lead Data Scientist-> CTO
- AI Background:
 - Manufacturing & supply chain analytics at Honeywell
 - Customer retention and targeting in automotive industry
 - Computer vision for geospatial & high performance computing (HPC) at GDIT
 - CV & NLP solutions for government agencies
 - Forward deployed engineer at AI start-up (federal focus)
 - Program manager & CTO overseeing multiple contracts



Advancing Lightweight Breast Cancer Detection Models: Parameter-Efficient Fine-Tuning and Knowledge Distillation from Foundational Models



Efficiency Comparison of Teacher and Student Models

Model	F1	Prec.	Recall	Dice	Total Params	% of Teacher	Avg. Time (ms)
SAM2	0.8842	0.9231	0.8485	0.6400	224,430,130	—	49.93
ViT-Large	0.9016	0.9255	0.8788	0.6917	25,083,605	0.11	18.22
ViT-Small	0.8586	0.8913	0.8283	0.5774	9,378,745	0.04	18.86
ResNet50	0.8404	0.8876	0.7980	0.6107	28,550,885	0.13	17.59
ResNet101	0.8065	0.8621	0.7576	0.6044	47,543,013	0.21	24.35

Presented at the Applied Machine Learning Conference in April 2026

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Course Overview

**Syllabus for
SEAS 8515-D6A
Data Engineering for AI
Summer 2026**

Instructor: Dr. Steven Cocke
eMail: steven.cocke@email.gwu.edu
Credit Hours: 4 credit hours
Course Website: On Blackboard

Class Time and Dates:

- Day and Time: Saturdays from 9:00am-12:00pm (Eastern)
- Class Meeting Dates: May 16, 23, 30 - Jun. 6,13,20,27, Jul. 4 (asynch),11,18,25, Aug. 1, 8, 15 (final exam)
- Attendance is expected at all sessions. If an absence from a class meeting is needed (due to family/medical or work-related emergency) students must contact the instructor in advance.
- Online classes are conducted via Zoom; Links are provided in Blackboard.
- Zoom link for Office Hours: gwu-edu.zoom.us/j/5812175257

Office Hours: For 3 hours every week I will be available for drop-in office hours, as follows:

- Every Tuesday, 7:00-8:30pm ET
- Every Sunday, 7:00-8:30pm ET

Bulletin Description of the Course:

Developing Python scripts to automate data pipelines, data ingestion, data processing, and data warehousing. Machine learning applications with Python including text mining and time series analysis.

Course Learning Objectives:

Upon completing the course, students will know how to:

- Perform data ingestion using Spark, Python and SQL for structured, unstructured, and streaming data.
- Develop and automate data pipelines for ETL processes, including data cleaning and preparation for AI model training.
- Apply machine learning methods for data analysis, including regression, classification, and anomaly detection.
- Implement and orchestrate scalable, production-level machine learning workflows and data pipelines to support AI analytics and business analytics.

Textbook

O'Reilly – GWU Resource
<https://learning.oreilly.com/>

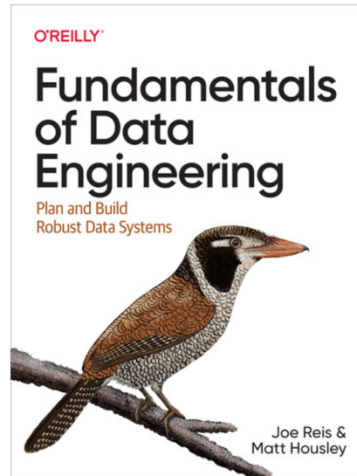
Available at GWU Library, link below:
https://wrlc-gwu.primo.exlibrisgroup.com/permalink/01WRLC_GWA/1j51gk4/alma99186401904504107



Fundamentals of Data Engineering

★★★★☆ 42 reviews

By [Joe Reis](#), [Matt Housley](#)



TIME TO COMPLETE:
13h 34m

LEVEL:
Beginner

SKILLS:
[Data Engineering](#)

PUBLISHED BY:
[O'Reilly Media, Inc.](#)

PUBLICATION DATE:
June 2022

PRINT LENGTH:
450 pages

+ Add to playlist

[Audiobook available](#)

Continue

Data engineering has grown rapidly in the past decade, leaving many software engineers, data scientists, and analysts looking for a comprehensive view of this practice. With this practical book, you'll learn how to plan and build systems to serve the needs of your organization and customers by evaluating the best technologies available through the framework of the data engineering lifecycle.

- Other Material: Databricks, Google Colab/Python IDE and Python for Data Analysis, 3rd Edition;
<https://learning.oreilly.com/library/view/python-for-data/9781098104023/>

Other References

- Will be called out at the beginning of each lecture
- Some topics don't have great textbooks and there might be a blog or online article that I reference that explains the information better

Administrative Comments

- Please email me at steven.cocke@gwu.edu (**please do not send emails thru blackboard**) and include SEAS8515 in email subject.
- Class attendance is automatically captured and reported (**prior notice is required for not being able to attend with valid reason**).
- The lecture notes/discussions/assignments are in scope for exams.

Administrative Comments: Assignments

- Assignments should be submitted on time (due 11:59PM EST evening before the next class (Friday nights)).
- **Late submissions are not accepted!**
- All submissions are individual work deliverables.
- Assignments should be submitted in the following format:
- **HW#_LastName_FirstName**
- **The HW# = the week #**
- Example for week 4 homework: **HW4_Cocke_Steven.ipynb**
- Please make sure you answer all the parts of the assignments as per the instructions.
- Assignments for this course will either be a hands-on coding assignment where the deliverable will be uploaded to Blackboard or a knowledge check (quiz) that will be completed through Blackboard
- All assignments will appear in the “Assignments” tab on Blackboard
- Some weeks you will have a choice between assignment options
- Please follow provided assignment templates when given
- Assignments are posted at 4pm ET every Saturday (except weeks of exams)

Administrative Comments: Grading

- My goal is always to have assignments that must be manually graded to be graded by the next week's lecture (see calendar example below)
- Some assignments are auto-graded (tests in blackboard like Assignment 1 this week) and will be returned sooner
- **If a week has gone by (7-8 days from submission)** and you are missing a grade, please reach out to me via email

JULY 2026

SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY
JUNE 2026 S M T W T F S 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30	AUGUST 2026 S M T W T F S 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31		1	2	3	4 Independence Day
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

Administrative Comments: Midterm and Final

- Occur during class time
- Two hours in length
- Midterm
 - Week 7
 - Content is from Weeks 1-5
- Final
 - Week 14
 - Content is from all weeks (except Week 13) with a focus on Weeks 6-12
 - Week 13 content is not on the final
- Material will come from:
 - The slides from lecture
 - The demonstrations from lecture
 - The weekly assignments
 - The assigned readings
- More detail to come when we get closer to the exam(s)

Overall Approach for the Course

- Both conceptual theory and hands-on examples will be used to convey information for topics.
- **Material for exam questions will come from class slides & lecture material, the course textbook, homework assignments, and assigned readings.**
- Lecture is comprised of a combination of slides and live hands-on examples or short videos.
- We will have one 10-15 minute break depending on the material for that day.
- Slides for the day will be posted on Blackboard at the start of class.
- Live demonstration or supplemental material from lecture will be posted after class.

Overall Comments on the Course

- Data Engineering is a broad and rapidly evolving discipline that spans data ingestion, storage, processing, transformation, orchestration, governance, and deployment
- Many of the topics covered in this course could each warrant an entire standalone course or specialization
- The goal of this course is to introduce foundational data engineering concepts, demonstrate practical implementations, expose you to modern tooling and workflows, and help prepare you for the research and implementation phases of the program
- The course emphasizes both conceptual understanding and hands-on application through coding exercises, demonstrations, and real-world workflows

Overall Comments on the Course

Throughout the course, we will:

- Use a textbook and lecture slides to introduce core concepts
- Use Python as the foundational programming language for data processing, automation, APIs, and machine learning workflows
- Use Apache Spark as a distributed data processing framework for scalable analytics and large-scale data engineering tasks
- Use Databricks as a modern cloud-based data engineering and analytics platform to gain familiarity with enterprise-style workflows
- Use SQL as a core language for querying, transforming, aggregating, and managing structured data
- Explore modern data engineering tooling and practices related to:
 - Containerization and reproducibility
 - Workflow orchestration and automation
 - Data pipelines and ETL/ELT processes
 - Streaming and batch processing
 - MLOps and experiment tracking
 - APIs and data integration
 - Data lakes, Delta Lake, and distributed storage systems
- Focus on understanding how these technologies work together within real-world AI and data-driven systems
- Build practical intuition that can be applied to research, software engineering, machine learning, and production AI workflows

THE GEORGE
WASHINGTON
UNIVERSITY

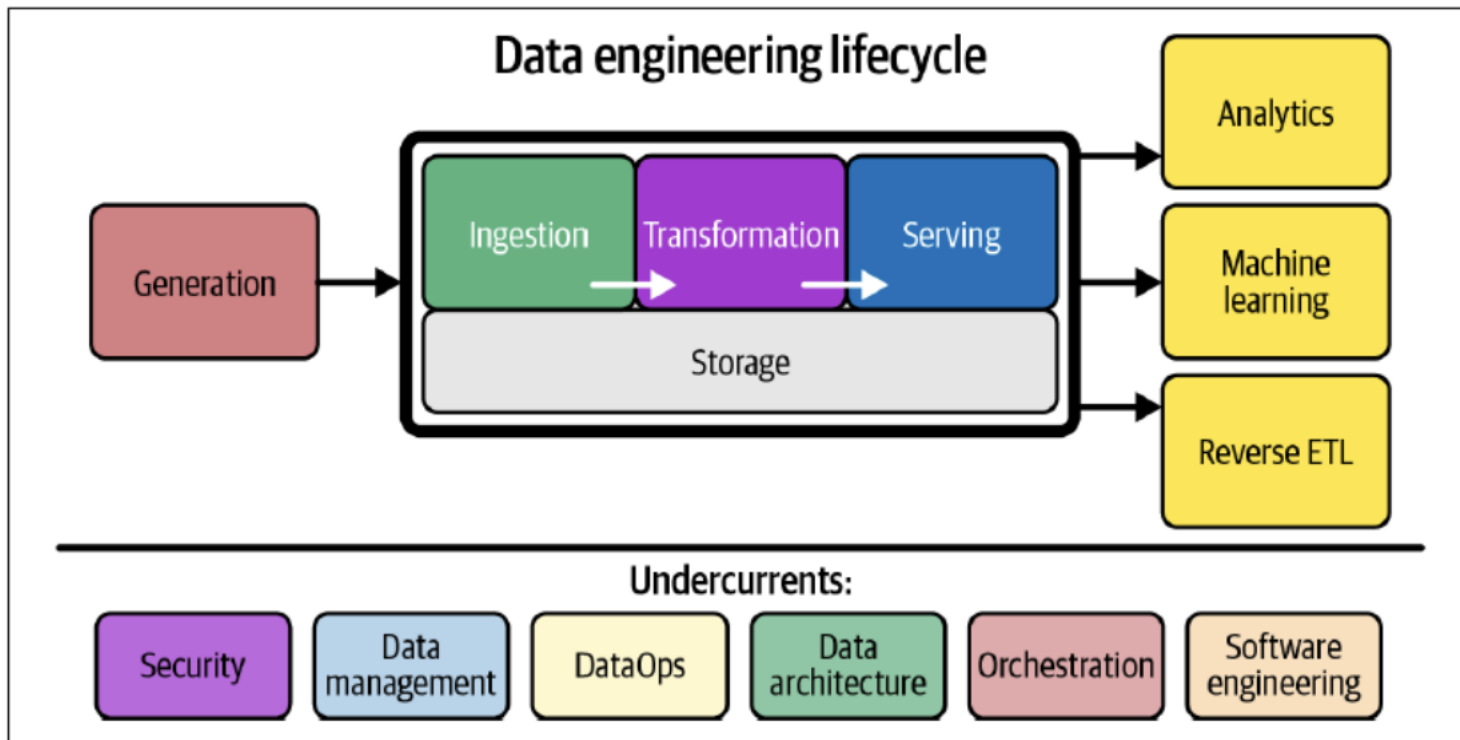
WASHINGTON, DC

Survey

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Python Foundations I



- Python has become the most popular coding language for Data Science & ML/AI research and production solutions
- It is also becoming more popular in Data Engineering and processes that control Data Engineering tasks

This course focuses on Data Engineering and hands-on examples/exercises will be in Python

Python Motivation

- Understanding Python syntax is essential for implementing data engineering and AI/ML solutions
- Python is the foundation of many state-of-the-art (S.O.T.A.) data and AI workflows
- From data ingestion and transformation → to feature engineering, model training, and deployment
- Python acts as the “glue” connecting storage systems, distributed compute, ML frameworks, and orchestration tools
- Most modern data engineering tools expose Python APIs, making it the common interface across the stack
- Strong Python fundamentals enable scalable, maintainable, and reproducible pipelines, not just one-off scripts

This course assumes Python not as a programming language, but as the primary interface to modern data and AI systems.

What Python is in this course

- Python is the primary interface to modern data systems and AI tools
- Used to orchestrate, connect, and automate data workflows
- Acts as the glue between storage, compute, and ML frameworks
- Ingests data from files, APIs, and streams
- Transforms and prepares data for analysis and ML
- Interfaces with distributed compute and warehouses
- Connects pipelines to models, dashboards, and services

Core Language Constructs

- Variables and dynamic typing enable rapid iteration
- Control flow defines how data moves through a pipeline
- Functions and Classes encapsulate logic and promote reuse
- Modules and packages organize growing codebases

```
1 threshold = 10
2
3 def classify(x):
4     if x > threshold:
5         return "high"
6     else:
7         return "low"
8
9 [classify(x) for x in [5, 12, 8, 20]]
```


Data Structures for Engineering Tasks

- Lists for ordered collections and iteration
- Tuples for fixed, immutable records
- Dictionaries for key–value and schema-like data
- Sets for uniqueness and membership testing

```
1 records = [  
2     {"id": 1, "value": 10},  
3     {"id": 2, "value": 15},  
4     {"id": 3, "value": 10},  
5 ]  
6  
7 unique_values = {r["value"] for r in records}  
8 unique_values|
```

Why Functions Matter

- Functions define repeatable pipeline steps
- Parameters allow pipelines to adapt to different data sources
- Modular design improves readability and testability
- Clear inputs and outputs improve reproducibility

```
def normalize(values, scale=1.0):  
    max_val = max(values)  
    return [(v / max_val) * scale for v in values]  
  
normalize([10, 20, 30])
```

Notebooks vs. Scripts vs. Packages

- Notebooks support exploration and experimentation
- Scripts enable automation and repeatability
- Packages support reuse, testing, and collaboration
- Mature projects often evolve through all three

```
1 def main():  
2     print("Running pipeline")  
3  
4 if __name__ == "__main__":  
5     main()
```

Working with Files, Paths, and Storage

- Understanding local vs. cloud storage is critical
- File paths differ across operating systems and environments
- Structured vs. unstructured data require different handling
- Poor path management is a common source of pipeline failure

```
1 from pathlib import Path
2
3 data_path = Path("data/input.txt")
4
5 if data_path.exists():
6     content = data_path.read_text()
7     print(content)
```

Errors, Exceptions, Debugging

- Understanding local vs. cloud storage is critical
- File paths differ across operating systems and environments
- Structured vs. unstructured data require different handling
- Poor path management is a common source of pipeline failure

```
1 def divide(a, b):  
2     try:  
3         return a / b  
4     except ZeroDivisionError as e:  
5         raise ValueError("Invalid denominator") from e  
6  
7 divide(10, 0)
```

Environments and Dependencies

- Python versions affect behavior and compatibility
- Libraries and dependencies must be managed explicitly
- Reproducibility is critical for research and production
- Environments must be portable across systems

```
1 !pip install pandas numpy scikit-learn
2 !pip freeze > requirements.txt
3 !pip install -r requirements.txt|
```

Performance and Scale Awareness

- Python often orchestrates rather than executes heavy computation
- Vectorized operations outperform explicit loops
- Distributed systems handle scale; Python coordinates them
- Knowing when to hand off work is a key engineering skill

```
1 import numpy as np
2
3 data = np.random.rand(1_000_000)
4
5 # fast
6 data.mean()
7
8 # slow
9 sum(data) / len(data)
```

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Python Notebook Demonstration

What To Work On This Week

- Homework #1 (Knowledge Check)
- Finish reviewing the notebook from class today.
- No reading this week. Take this time to review Python concepts, attend office hours if help is needed.